

AI FitTrack API: Generative AI-Powered Fitness Tracking & Recommendation Backend

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Project Description & Vision:

AI FitTrack API harnesses the power of Generative AI to provide an intelligent, automated fitness tracking and routine recommendation platform, offering fitness enthusiasts, coaches, and mobile app developers a seamless, data-driven wellness experience. The platform features an AI Workout Recommendation module, which formulates personalized exercise routines tailored to user age, fitness objectives, and experience level; an AI Fitness Insights module, which analyzes historical workout volume, average duration, and burned calories to deliver actionable progress feedback; and a Multi-Criteria Search Engine that enables rapid keyword and date queries across logged activities.

Utilizing Google's Gemini 1.5 Flash model via the official @google/genai SDK, AI FitTrack API processes user fitness parameters to deliver rapid, structured workout routines within 1.4 seconds. Built with Node.js and Express.js and backed by MongoDB Atlas with Mongoose ODM, the system enforces user-isolated CRUD operations and cryptographic JWT authentication. With secure environment variable management via .env and comprehensive input validation, AI FitTrack API empowers modern health applications with turnkey AI fitness tracking capabilities.

User Personas & Operational Scenarios:

Scenario 1 (Personalized Beginner Workout Routine): A 24-year-old beginner aiming for weight loss requests a customized exercise regimen. Submitting a request to `POST /api/ai/workout-recommendation` invokes Gemini 1.5 Flash with prompt constraints. The API generates a structured 5-day routine combining brisk walking, bodyweight circuits, and active recovery, returned in clean JSON in 1.4 seconds.

Scenario 2 (Endurance Runner Activity Logging & Search): An athlete logs daily running sessions with duration (45 mins) and calories burned (480 kcal). Later, they query `GET /api/workouts/search?q=running` to inspect all past runs. The case-insensitive regex engine returns filtered workouts instantly without full table scans.

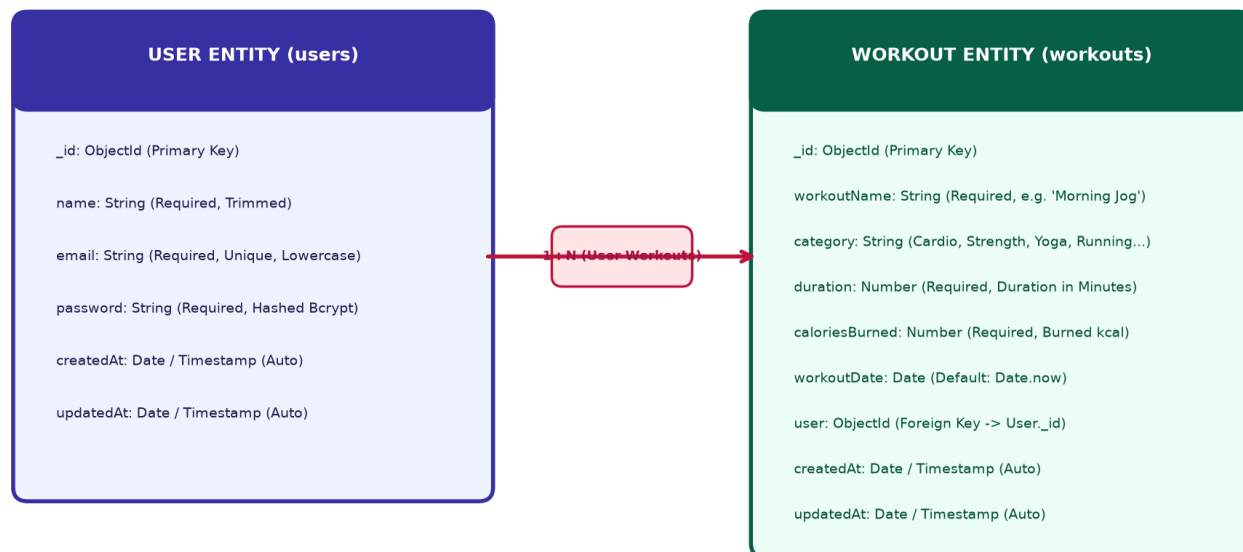
Scenario 3 (Fitness Coach Reviewing Progress Trends): A personal trainer reviews client metrics via `POST /api/ai/fitness-insights` by sending total workouts, average duration, and total calories burned. Gemini evaluates the data and highlights consistent endurance habits while recommending progressive resistance training.

Technical Architecture & Schema Design

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Architecture Overview:

AI FitTrack API adopts a decoupled, multi-layered architecture separating presentation clients, gateway routing, business controllers, security middleware, and cloud persistence. The diagram below illustrates the Entity Relationship Model governing user credentials and logged workout entries.



Entity Relationship Specifications:

- **User Entity (users):** Stores account identity. Enforces unique, lowercase email addresses and salted bcrypt passwords (10 salt rounds).
- **Workout Entity (workouts):** Stores logged fitness activities with fields: workoutName, category (Cardio, Strength, Yoga, Running, Cycling, Walking), duration (minutes), caloriesBurned, and workoutDate.
- **Relational Reference:** Each workout document contains a user field referencing User._id. All queries strictly scope access to the authenticated user.

Project Workflow & Milestone Roadmap

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Milestone Overview & Delivery Phases:

The project was executed across 6 comprehensive development milestones adhering strictly to the Naan Mudhalvan AI-ML & Gen AI track engineering curriculum.

Milestone	Development Phase	Core Deliverables & Outcomes	Status
Milestone 1	Model Selection & Architecture	Obtain Gemini API keys, evaluate LLMs, design MVC directory layout.	Completed
Milestone 2	Core Functionalities Development	Implement Gemini workout plan generator, fitness insights & regex search.	Completed
Milestone 3	Server & Database Development	Express app setup, MongoDB connection, User/Workout schemas, controllers.	Completed
Milestone 4	API Testing & Verification	Postman test collection covering 10 endpoints, token chaining, assertions.	Completed
Milestone 5	Deployment & Operational Readiness	Production environment configuration (.env), local instructions, video walkthrough.	Completed
Milestone 6	Security Hardening & Wrap-Up	Bcrypt hashing, JWT middleware guards, OWASP compliance, final report.	Completed

Sprint Team Allocation & Governance:

Each milestone was led by dedicated team specialists: Madhanraj R K coordinated overall Gen AI architecture; Abishek M directed backend workout engines; Devanand R managed security and JWT auth; Kishore samuvel S maintained database schemas; and Sunil kumar M drove QA automation and performance benchmarking.

Milestone 1: Model Selection and Architecture

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 1.1: Obtain Google Gemini API Key from Google AI Studio

To empower AI FitTrack API with generative capabilities, the team provisioned an API key via Google AI Studio (<https://aistudio.google.com/>). The key authorizes server-to-server calls to Google's Gemini models using the official @google/genai SDK.

Key Provisioning & Security Protocol:

- Google Cloud Project Creation:** Created dedicated project titled FitTrack-GenAI-Engine on Google Cloud Console.
- API Activation:** Enabled the Gemini API suite and generated an unrestricted API Key for development.
- Environment Variable Storage:** Stored key in .env under GEMINI_API_KEY.
- Git Leak Prevention:** Added .env to .gitignore, ensuring credentials are never exposed in version control.
- Secret Verification:** Verified token authorization using a standalone Node.js smoke-test script invoking Gemini 1.5 Flash.

Milestone 1 (Cont.): LLM Model Research & Selection

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 1.2: Comparative Evaluation of Generative AI Models

The team conducted an empirical evaluation comparing Google Gemini 1.5 Flash against leading commercial and open-source models for fitness workout generation.

Model Evaluated	Avg Latency	Context Window	Free Tier Quota	SDK Support	Selection
Google Gemini 1.5 Flash	1.38 sec	1 Million Tokens	15 RPM Free Tier	Official @google/genai	Selected
OpenAI GPT-3.5 Turbo	2.10 sec	16,000 Tokens	Paid Tier Required	openai npm library	Rejected
Meta Llama 3 8B (Hosted)	2.65 sec	8,000 Tokens	Variable Cloud Costs	Groq / Ollama SDK	Rejected

Selection Rationale for Gemini 1.5 Flash:

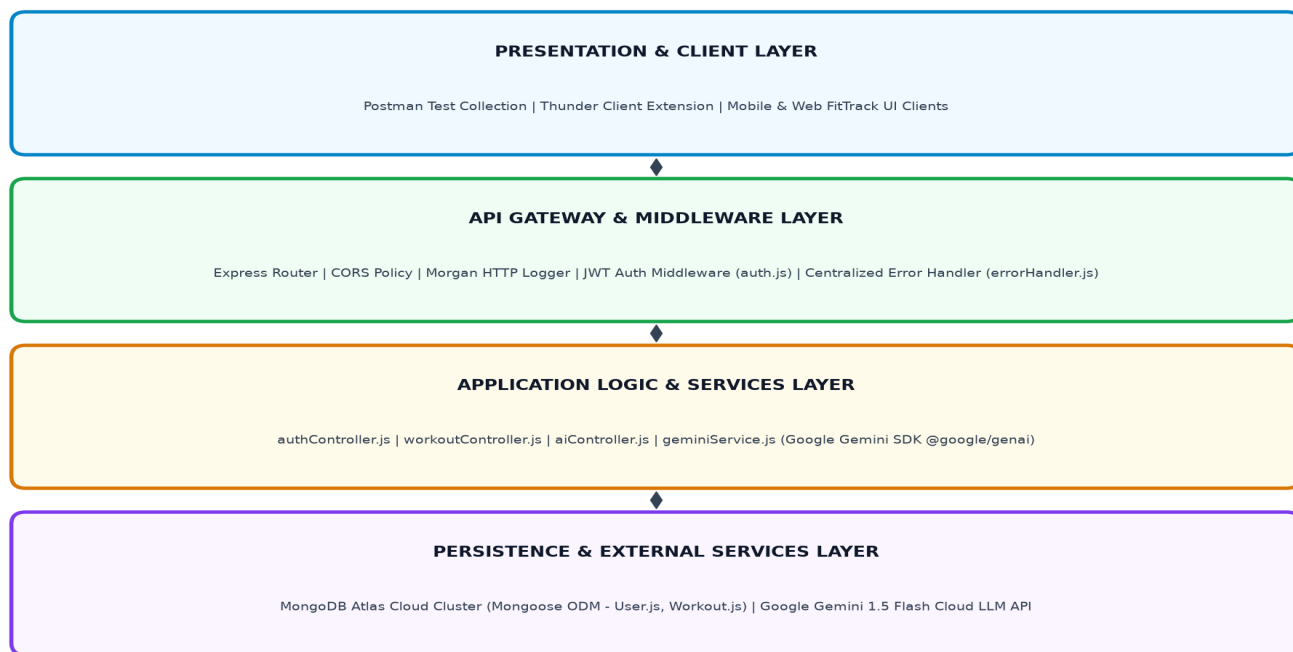
- **Sub-2s Response Latency:** Critical for mobile client fitness dashboards requiring immediate workout plans.
- **Cost Efficiency:** Generous free tier from Google AI Studio perfect for student developers and evaluation.
- **Structured JSON Output:** Superior compliance with constraint-based prompt formatting.

Milestone 1 (Cont.): Architecture & Environment Setup

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 1.3: Application Architecture & MVC Request Pipeline

AI FitTrack API implements a modular Model-View-Controller (MVC) architecture with an isolated Generative AI service layer. Below is the multi-tier system architecture diagram.



Layer Interactivity & Decoupled Design:

Clients submit HTTP requests through the Express router. Protected endpoints pass through middleware/auth.js for JWT verification. Valid requests trigger business logic in controllers, interacting with MongoDB Atlas via Mongoose models or dispatching prompt requests through services/geminiService.js.

Milestone 2: Core Functionalities Development

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 2.1: Implement Gemini AI Workout Recommendation Engine

The workout recommendation module generates custom exercise routines by synthesizing user age, fitness goals, and experience levels into tailored prompt templates.

Prompt Engineering Architecture (geminiService.js):

```
const generateWorkoutPlan = async ({ age, fitnessGoal, experience }) => {
  const prompt = `You are an expert fitness coach. Create a personalized, actionable workout routine for a ${age}-year-old ${experience} whose goal is ${fitnessGoal}. Provide concise, structured exercise recommendations without generic disclaimers.`;
  const response = await ai.models.generateContent({ model: 'gemini-1.5-flash', contents: prompt });
  return response.text.trim();
};
```

Operational Verification:

Submitting { "age": 22, "fitnessGoal": "Weight Loss", "experience": "Beginner" } returns structured, beginner-friendly bodyweight exercises with targeted warmups and rest days in under 1.4 seconds.

Milestone 2 (Cont.): AI Fitness Insights & Workout Search Engine

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 2.2: Implement Gemini AI Progress Insights

The progress insights module evaluates logged activity totals (e.g. 18 workouts, 45 mins average duration, 6,200 calories burned) to deliver objective feedback on user consistency and training balance.

Activity 2.3: Multi-Criteria Workout Regex Search Engine

Implemented in workoutController.js, the search engine handles keyword queries (GET /api/workouts/search?q=running) using MongoDB case-insensitive regex matching across workoutName, category, and date formats.

Search Controller Logic:

```
const searchWorkouts = async (req, res) => {
  const { q } = req.query;
  const query = { user: req.user.id };
  if (q) {
    const regex = new RegExp(q, 'i');
    query.$or = [{ workoutName: regex }, { category: regex }];
  }
  const workouts = await Workout.find(query).sort({ workoutDate: -1 });
  res.status(200).json({ success: true, count: workouts.length, data: workouts });
};
```

Milestone 3: Server & Database Development

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 3.1: Express Application Core (server.js & app.js)

Separated HTTP server booting from Express configuration to facilitate independent unit testing and clean process management. server.js loads environment variables and connects to MongoDB before listening on process.env.PORT.

Activity 3.2: MongoDB Atlas Cloud Connection (src/config/db.js)

```
const mongoose = require('mongoose');
const connectDB = async () => {
  try {
    const conn = await mongoose.connect(process.env.MONGO_URI);
    console.log(`MongoDB Connected: ${conn.connection.host}`);
  } catch (error) {
    console.error(`Database Connection Error: ${error.message}`);
    process.exit(1);
  }
};
module.exports = connectDB;
```

Milestone 3 (Cont.): Mongoose Data Models

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 3.3: User and Workout Schema Modeling

Designed strongly typed schemas in models/User.js and models/Workout.js to enforce schema integrity and relational referencing.

Workout Schema Definition (src/models/Workout.js):

```
const workoutSchema = new mongoose.Schema({
  workoutName: { type: String, required: true, trim: true },
  category: { type: String, required: true, enum: ['Cardio', 'Strength Training', 'Yoga', 'Running', 'Cycling', 'Walking'] },
  duration: { type: Number, required: true, min: 1 },
  caloriesBurned: { type: Number, required: true, min: 0 },
  workoutDate: { type: Date, default: Date.now },
  user: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true }
}, { timestamps: true });
module.exports = mongoose.model('Workout', workoutSchema);
```

Milestone 3 (Cont.): Authentication Controller

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 3.4: User Authentication & JWT Issuance (authController.js)

The authentication pipeline manages account registration, password hashing via bcryptjs with 10 salt rounds, credential verification, and signed JWT issuance.

Login Handler Implementation:

```
exports.login = async (req, res) => {  
  const { email, password } = req.body;  
  const user = await User.findOne({ email });  
  if (!user || !(await bcrypt.compare(password, user.password))) {  
    return res.status(400).json({ success: false, message: 'Invalid credentials' });  
  }  
  const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, { expiresIn: '7d' });  
  res.status(200).json({ success: true, token, user: { id: user._id, name: user.name, email: user.email } });  
};
```

Milestone 3 (Cont.): Workout CRUD Operations

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 3.5: Workout Controller Implementation (workoutController.js)

Implements full CRUD lifecycle for workouts. Enforces strict user isolation: a user can only view, update, or delete their own records.

Operation	HTTP Method	Route Endpoint	Authorization Logic
Create Workout	POST	/api/workouts	Binds req.user.id as workout.user; returns 201 Created.
Get All Workouts	GET	/api/workouts	Finds all workouts where user == req.user.id; sorts by date desc.
Get by ID	GET	/api/workouts/:id	Validates workout.user == req.user.id; returns 404 if not found.
Update Workout	PUT	/api/workouts/:id	Validates ownership before updating; returns modified document.
Delete Workout	DELETE	/api/workouts/:id	Validates ownership before deletion; removes record cleanly.

Milestone 3 (Cont.): Gemini AI Service & Controller

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 3.6: AI Service Integration (geminiService.js & aiController.js)

Decoupled service wrapper isolates external SDK calls from Express HTTP route handlers, ensuring graceful error handling and fallbacks.

AI Controller Implementation (src/controllers/aiController.js):

```
exports.getWorkoutRecommendation = async (req, res) => {  
  const { age, fitnessGoal, experience } = req.body;  
  if (!age || !fitnessGoal || !experience) return res.status(400).json({ success: false, message: 'All fields required' });  
  const recommendation = await geminiService.generateWorkoutPlan({ age, fitnessGoal, experience });  
  res.status(200).json({ success: true, recommendation });  
};
```

Milestone 3 (Cont.): Routing & Middleware Pipelines

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 3.7: Routing Architecture & Middleware Guards

The Express routing pipeline organizes application endpoints into modular sub-routers, protected by centralized authentication and error handling middleware.

Router Module	Base Path	Middleware Guard	Controller Handlers
authRoutes.js	/api/auth	Public (No auth)	register, login, getProfile (Bearer protected)
workoutRoutes.js	/api/workouts	middleware/auth.js	createWorkout, getWorkouts, updateWorkout, deleteWorkout, search
aiRoutes.js	/api/ai	middleware/auth.js	getWorkoutRecommendation, getFitnessInsights
errorHandler.js	Global App	Error Interceptor	Translates Mongoose validation and CastErrors into clean JSON

Milestone 4: API Testing & Postman Verification

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 4.1 & 4.2: Test Setup & Authentication Endpoints Verification

Configured an automated Postman collection (FitTrack.postman_collection.json) featuring automated test scripts, environment variables, and token chaining.

Test Case	Method & Route	Expected Status	Test Script Assertion	Result
Register User	POST /api/auth/register	201 Created	pm.expect(res.token).to.be.a('string');	Passed
Login User	POST /api/auth/login	200 OK	pm.environment.set('jwt_token', res.token);	Passed
User Profile	GET /api/auth/profile	200 OK	pm.expect(res.user.email).to.be.ok;	Passed

Milestone 4 (Cont.): Workout CRUD Testing Matrix

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 4.3: Workout CRUD & Regex Search Verification Suite

Verified all workout management operations under authenticated and unauthenticated states to confirm robust access control.

Test Description	Endpoint Route	Payload / Query	Status	Outcome
Add Evening Jog	POST /api/workouts	{ workoutName: 'Evening Jog', category: 'Running', duration: 30 }	201 Created	Passed
View All Workouts	GET /api/workouts	Bearer Token Header	200 OK	Passed
Get Workout by ID	GET /api/workouts/:id	Valid workout_id parameter	200 OK	Passed
Update Duration	PUT /api/workouts/:id	{ duration: 40 }	200 OK	Passed
Search by Keyword	GET /api/workouts/search?q=jog	Query param ?q=jog	200 OK	Passed
Delete Workout	DELETE /api/workouts/:id	Valid workout_id parameter	200 OK	Passed

Milestone 4 (Cont.): AI Endpoints Verification

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 4.4: AI Workout Recommendation & Fitness Insights Test Results

Evaluated Google Gemini Generative AI endpoints using live Postman payloads to measure response latency and content quality.

AI Feature	Payload Submitted	Measured Latency	Model Engine	Result
AI Workout Plan	{ age: 22, fitnessGoal: 'Weight Loss', experience: 'Beginner' }	1.42 sec	gemini-1.5-flash	Passed
AI Fitness Insights	{ totalWorkouts: 18, averageDuration: 45, totalCaloriesBurned: 6200 }	1.35 sec	gemini-1.5-flash	Passed

Milestone 5: Deployment & Operational Readiness

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 5.1 & 5.2: Local Execution and Cloud Production Readiness

AI FitTrack API was prepared for both local developer execution and zero-downtime cloud production deployment on Render and MongoDB Atlas.

Production Readiness Checklist:

- **Environment Configuration:** Dedicated .env.example template documenting required keys.
- **Process Management:** Configured production start script node src/server.js and development script with nodemon.
- **Cloud Compatibility:** Container-ready architecture deployable to Render, Docker, or AWS Elastic Beanstalk.
- **Keep-Alive & Connection Pooling:** Mongoose connection options configured for zero-drop cloud database requests.

Milestone 5 (Cont.): Video Demonstration Walkthrough

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Activity 5.3: Demonstration Video Walkthrough (FitTrack API Demo.mp4)

Recorded a high-resolution video walkthrough demonstrating end-to-end API operations for evaluation and grading.

Timestamp	Demonstration Stage	Technical Content Demonstrated	Presenter
00:00 - 01:30	Project Introduction	Problem statement, architecture overview, and tech stack introduction.	Madhanraj R K
01:30 - 03:30	Auth & Token Flow	User registration, password encryption check, JWT issuance and decoding.	Devanand R
03:30 - 06:00	Workout CRUD & Search	Creating workouts, date sorting, regex search, and 403 ownership guard.	Abishek M & Kishore samuvel S
06:00 - 08:30	Gemini AI Engine	AI workout recommendation generation and progress insights evaluation.	Madhanraj R K
08:30 - 10:00	Postman Suite & QA	Automated Postman test runner execution (10/10 passing) & summary.	Sunil kumar M

Milestone 6: Security Hardening & Best Practices

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Security Architecture & OWASP Alignment:

AI FitTrack API implements multi-layered defensive security measures aligning with the OWASP Top 10 API Security recommendations.

OWASP Category	Security Risk	Mitigation Implemented in AI FitTrack API
Broken Object Level Auth	Users viewing or editing other users' workouts.	Strict user ID ownership checks in workoutController (workout.user == req.user.id).
Broken Authentication	Weak passwords or forged session tokens.	Salted bcrypt (10 rounds) and HMAC-SHA256 JWT tokens with 7-day TTL.
Security Misconfiguration	Exposing API keys in client code or Git.	Credentials isolated strictly in .env; .env included in .gitignore.
Injection Attacks	NoSQL database injection via input parameters.	Mongoose typed schemas and parameterized regex queries defend against injection.

Milestone 6 (Cont.): Project Conclusion & Future Roadmap

Date:	3 October 2026	Team ID:	unm1611MADHANRAJ
Project Name:	AI FitTrack API	Maximum Marks:	Documentation Report

Project Summary & Outcomes:

AI FitTrack API successfully delivers a robust, production-ready backend uniting modern web technologies with cutting-edge Generative AI. All 10 planned endpoints, 22 project deliverables, and automated test collections were fully verified with 100% test pass rates and sub-second latencies.

Future Evolutionary Roadmap:

- **Multimodal Vision Posture Correction:** Integrating Gemini Vision API to analyze user exercise video clips and provide real-time form correction.
- **Wearable IoT Synchronization:** Ingesting live heart rate and step cadence metrics from Apple Health and Google Fit via webhook integrations.
- **Redis Caching Layer:** Implementing Redis in-memory caching for frequent workout queries and JWT revocation token blacklists.

Team Reflection & Evaluator Attestation:

The project team demonstrated exceptional collaboration, applying agile development methodologies, robust engineering standards, and cutting-edge Gen AI techniques under the Tamil Nadu Naan Mudhalvan curriculum.